



國立臺北科技大學

資訊工程系碩士班

碩士學位論文

**基於 FruityMesh 強化藍牙低功耗網狀網路
韌性與服務品質**

**Resilience and Service Enhancement in FruityMesh
Based Bluetooth Low Energy Mesh Networks**

研究生：張博勳

指導教授：吳和庭 博士

中華民國一百一十五年七月

摘要

關鍵詞：藍牙低功耗、物聯網、FruityMesh、BLE Mesh、QoS、拓樸恢復、非揮發性歷史鄰居快取、優先權調度

藍牙低功耗 (Bluetooth Low Energy, BLE) 具備低成本、低功耗與易於部署等特性，已廣泛應用於物聯網 (Internet of Things, IoT) 場景下的無線感測網路 (Wireless Sensor Networks, WSN)。隨著節點部署規模擴大，BLE Mesh 網路需同時面對多節點資料傳輸、有限的無線頻寬與節點斷連等挑戰。既有研究已透過目的導向傳輸 (Destination-Oriented Transmission, DOT) 與階層式連線間隔 (Connection Interval, CI) 調整，改善了 FruityMesh 的封包傳遞路徑並緩解根節點周邊的壅塞情形。然而，當匯集節點 (Sink) 發生異常斷連時，系統恢復速度仍然緩慢，且在高流量情境下，即便調整 CI 參數仍難以完全負荷突發的傳輸壓力，這顯示系統的重連效率與對關鍵資料的傳輸品質仍有優化空間。

針對上述限制，本論文提出一套提升 FruityMesh 服務穩定性與拓樸恢復的優化方案。在拓樸恢復方面，針對 Sink 節點重連延遲較長的瓶頸，設計基於非揮發性歷史鄰居快取 (Non-Volatile Historical Neighbor Cache) 的重連機制，透過節點間過往的連線資訊，於掃描過程中將接收到的封包即時比對歷史鄰居紀錄表，這使節點能更快速辨識出可靠的連線對象，進而縮短了無效掃描與決策等待的時間。在資料傳輸方面，本論文深入分析 FruityMesh 的服務品質 (Quality of Service, QoS) 機制，並整合其內建的優先權管理功能，當網路流量達到飽和，且單靠 CI 配置無法有效排除壅塞時，透過優先權調度機制，使高優先權 (High Priority) 封包能優先取得傳輸資源，降低較低優先權流量對關鍵資料的影響，保障核心訊息的通訊品質。

實驗結果顯示，本機制能顯著縮短平均重連時間。在 1,200 筆封包的壓力測試下，即使總流量超出頻寬上限，高優先權封包抵達率 (PDR) 仍維持 99% 以上，並顯著降低傳輸延遲，有效改善了 FruityMesh 在節點重連、高流量競爭與頻寬受限情境下的服務穩定性與拓樸恢復效率。

ABSTRACT

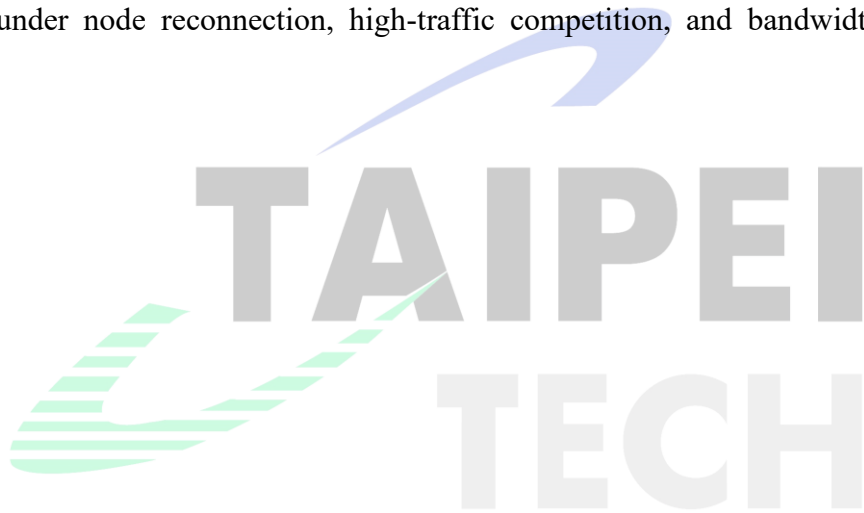
Keywords: Bluetooth Low Energy (BLE), Internet of Things, FruityMesh, BLE Mesh, Quality of Service (QoS), Topology Recovery, Non-Volatile Historical Neighbor Cache, Priority Scheduling

Bluetooth Low Energy (BLE) features low cost, low power consumption, and ease of deployment, has been widely applied to wireless sensor networks (WSNs) in Internet of Things (IoT) scenarios. As the scale of node deployment increases, BLE Mesh networks must simultaneously address challenges such as multi-node data transmission, limited wireless bandwidth, and node disconnections. Previous studies have improved the packet delivery paths of FruityMesh and alleviated congestion around the root node through Destination-Oriented Transmission (DOT) and hierarchical Connection Interval (CI) adjustment. However, when a Sink node experiences an abnormal disconnection, the system recovery process remains slow. In high-traffic scenarios, even with CI parameter adjustment, the network still struggles to fully handle burst transmission loads, indicating that there is still room for improvement in reconnection efficiency and the transmission quality of critical data.

To address these limitations, this thesis proposes an optimization scheme to improve the service stability and topology recovery of FruityMesh. In terms of topology recovery, this thesis focuses on the bottleneck of long reconnection delays for Sink nodes and designs a reconnection mechanism based on a Non-Volatile Historical Neighbor Cache. By utilizing previous connection information among nodes, the proposed mechanism compares received packets with the historical neighbor record table in real time during the scanning process. This enables nodes to identify reliable connection candidates more quickly, thereby reducing ineffective scanning and decision-making delays. In terms of data transmission, this thesis analyzes the Quality of Service (QoS) mechanism of FruityMesh and integrates its built-in priority management

function. When network traffic reaches saturation and CI configuration alone cannot effectively resolve congestion, a priority scheduling mechanism enables high-priority packets to preferentially secure transmission resources. This reduces the impact of lower-priority traffic on critical data and ensures the communication quality of core messages.

Experimental results show that the proposed mechanism can significantly reduce the average reconnection time. In a stress test involving 1,200 packets, even when the total traffic exceeded the bandwidth limit, the Packet Delivery Ratio (PDR) of high-priority packets remained above 99%, while transmission latency was significantly reduced. Overall, the proposed scheme effectively improves the service stability and topology recovery efficiency of FruityMesh under node reconnection, high-traffic competition, and bandwidth-constrained scenarios.



誌謝



目錄

摘要.....	i
ABSTRACT.....	ii
誌謝.....	iv
目錄.....	v
圖目錄.....	vi
表目錄.....	vii
第一章 緒論.....	1
1.1 研究背景.....	1
1.2 研究動機與目的.....	2
1.3 論文架構.....	3
第二章 相關技術與背景.....	5

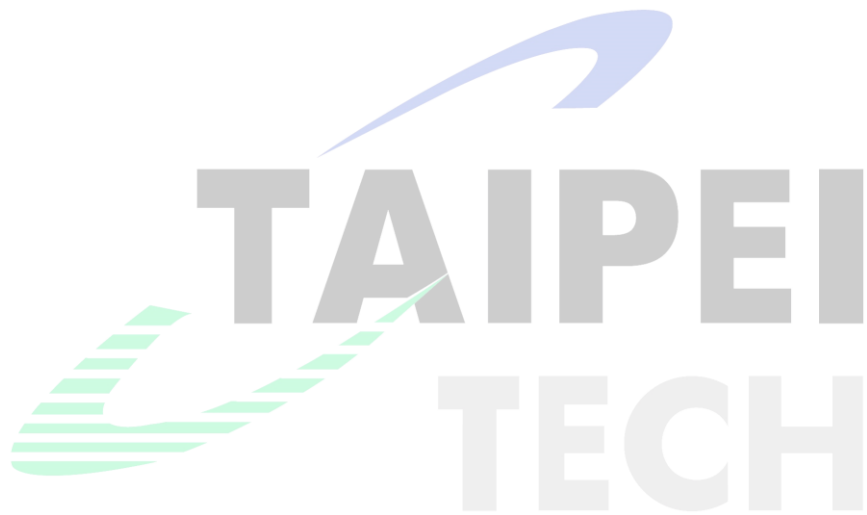


圖目錄

圖 1.1 2025 年至 2030 年全球物聯網市場規模與成長趨勢圖[1]	1
--	---



表目錄



第一章 緒論

1.1 研究背景

隨著感測技術、嵌入式系統與無線通訊協定的成熟，物聯網（Internet of Things, IoT）已成為推動智慧城市、工業 4.0 及智慧化生活的核心技術。根據市場研究機構 Mordor Intelligence 的數據顯示（如圖 1.1 所示），全球物聯網市場規模預計從 2025 年的 1.35 兆美元，以 15.04% 的複合年增長率（CAGR）持續擴張，至 2030 年將達到 2.72 兆美元之規模。其中亞太地區被評為成長最快速的市場，顯示出該領域在區域產業升級中的關鍵地位，而如此龐大的市場規模亦意味著連網設備的密度與數據量將呈現爆發式成長，進而對底層通訊架構的穩定性提出了更高要求。

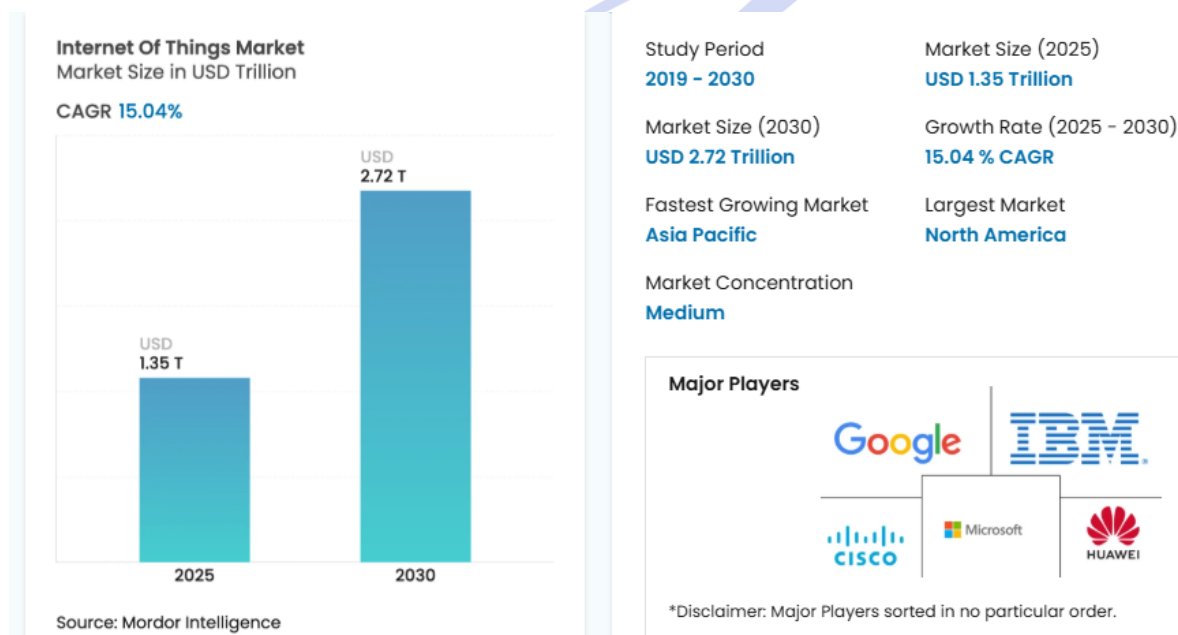


圖 1.1 2025 年至 2030 年全球物聯網市場規模與成長趨勢圖[1]

在實現大規模設備連接的過程中，無線感測網路（Wireless Sensor Networks, WSN）扮演了重要角色。特別是在覆蓋範圍廣大的監測環境下，由於單一節點的傳輸距離有限，網路必須透過多跳（Multi-hop）路徑，將數據由感測器逐跳轉傳至網關。然而，在這種多層級的結構下，網路的複雜度會隨節點數量大幅增加，導致在

封包衝突管理、路徑損耗以及時間同步等方面面臨挑戰。任何中間節點的失效，都可能引發整體網路的中斷。

針對低功耗與大規模部署的需求，藍牙低功耗 (Bluetooth Low Energy, BLE) 技術具備省電與高相容性等優勢，已從傳統的點對點連線發展至支援網狀網路 (Mesh Networking) 架構。儘管 BLE Mesh 能有效擴大通訊範圍，但在實際的變動環境與高負載應用中，如何在維持低功耗的同時確保通訊的高穩定性，並在網路受到干擾時達成快速的自我修復與重建，仍是目前大規模物聯網部署中需要解決的重要問題。

1.2 研究動機與目的

在藍牙低功耗網狀網路的研究中，FruityMesh 由於具備多跳傳輸、節點自動連線與拓樸自我修復等特性，常被用於 BLE Mesh 相關系統之開發與驗證。然而，隨著節點數量增加，網路中的封包轉送、角色切換與資料匯流負載也會隨之提高，進而影響整體傳輸效率與服務穩定性。因此，如何在低功耗與多節點環境下維持穩定通訊，成為 FruityMesh 應用中重要的研究議題。

針對 FruityMesh 傳輸效能改善，前人研究已分別從封包傳輸方向、連線角色切換、拓樸建立與連線參數調整等面向進行優化。溫淇淼 [3] 提出 DOT 與 DOST 機制，改善 FruityMesh 在多跳傳輸中可能產生的重複傳輸與角色衝突問題；陳柏勳 [2] 則進一步改善拓樸建立流程，使 Sink 節點能維持 Root 角色，並透過階層式 Connection Interval (CI) 調整機制，緩解靠近 Sink 節點的資料壅塞問題。上述方法已有效提升 FruityMesh 的封包傳輸效率與網路穩定性，但在節點重啟、Sink 異常斷線，以及高流量傳輸情境下，仍存在進一步改善空間。

首先，在拓樸恢復方面，當節點發生 reset 或 Sink 重新加入網路時，系統仍需重新進行掃描、鄰近節點收集、連線選擇與握手流程。即使節點過去曾與穩定鄰居建立連線，原生流程仍主要依賴重新搜尋與即時決策，因此可能造成較長的拓樸恢復時間。其

次，在高流量情境下，當封包數量超過可用頻寬時，即使透過 CI 調整增加部分節點的傳輸機會，重要封包仍可能與一般封包競爭相同傳輸資源，使關鍵資料的延遲與抵達率受到影響。

基於上述問題，本研究主要目的分為兩部分。第一，設計非揮發性歷史鄰居快取（Non-Volatile Historical Neighbor Cache）機制，保存節點過去穩定連線之鄰居資訊，使節點於 reset 後能優先參考歷史紀錄進行重連決策，藉此縮短無效掃描與等待時間，提升拓樸恢復效率。第二，應用 FruityMesh 內建之優先權調度機制，將不同重要性的封包進行分級，使高優先權封包在網路流量飽和時仍能優先取得傳輸機會，以提升關鍵資料於高流量環境下的服務品質。

本研究將透過 CherrySim 模擬環境與 Nordic nRF52840-DK 實體平台進行驗證，並以拓樸重建時間、封包抵達率（Packet Delivery Ratio, PDR）與平均封包延遲（Average Packet Delay）作為主要評估指標。透過與前人研究成果進行比較，驗證本研究所提出之歷史重連機制與優先權應用方式，是否能在節點重啟與高流量傳輸情境下，有效提升 FruityMesh 網路韌性與服務品質。

1.3 論文架構

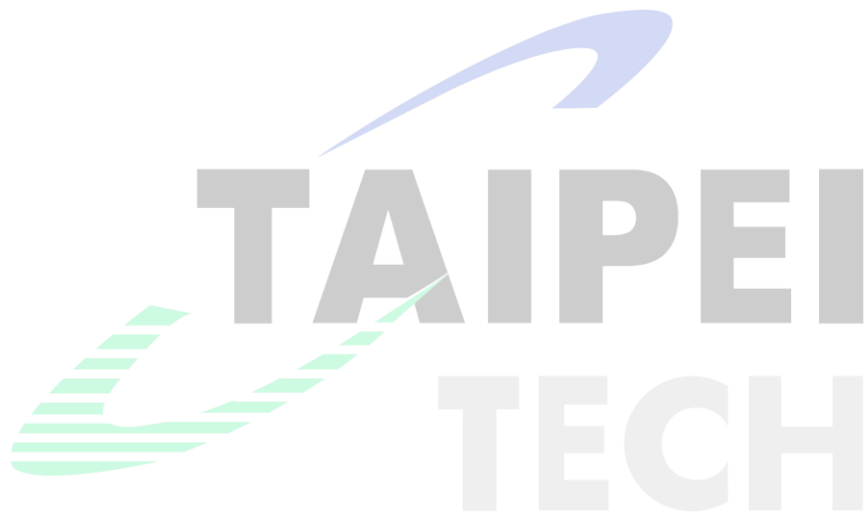
本論文共分為五章，第一章介紹物聯網、無線感測網路與 BLE Mesh 的發展背景，並說明本研究欲改善 FruityMesh 在拓樸恢復效率與高流量傳輸品質上的問題。

第二章介紹本研究所需之相關技術與背景，包含 BLE 通訊機制、TDMA、Bluetooth Mesh、Nordic nRF52840-DK、FruityMesh 網路形成與恢復機制，並整理 DOT、DOST、Sink-oriented 拓樸建立與階層式 CI 調整等 FruityMesh 傳輸效能改善方法。

第三章說明本研究提出之非揮發性歷史鄰居快取重連機制，並介紹 FruityMesh 原生優先權機制於高流量情境下的應用方式。

第四章透過 CherrySim 模擬環境與 Nordic nRF52840-DK 實體平台進行實驗，並以拓樸恢復時間、封包抵達率與平均封包延遲作為評估指標。

第五章總結本研究結果，並提出未來可延伸之研究方向。



第二章 相關技術與背景

2.1 Bluetooth Low Energy (BLE)

藍牙技術聯盟（Bluetooth Special Interest Group, 簡稱 Bluetooth SIG）成立於 1998 年，是一個負責監督藍牙技術發展並推動全球設備互通性的非營利同業公會，其在全球擁有超過三萬六千家企業會員共同致力於制定與維護藍牙相關規範，而隨著物聯網（Internet of Things, IoT）市場的急速擴張與感測設備部署需求的增加，如今已廣泛應用於穿戴式裝置（Wearable Devices）、智慧醫療與健康監測、智慧家庭自動化系統，以及工業物聯網（IIoT）感測節點等多元領域。

藍牙低功耗（Bluetooth Low Energy, BLE）是一種以低功耗、短距離無線通訊為主要目標的藍牙技術，相較於傳統藍牙，BLE 更適合用於少量資料與間歇性傳輸的應用情境，裝置在大部分時間可維持低功耗狀態，僅於需要廣播、掃描或傳輸資料時才喚醒運作，因此能有效延長電池使用時間，並降低裝置長時間部署時的維護成本。

2.1.1 BLE 通道架構與傳輸模式

在實體層（Physical Layer）架構上，BLE 運作於 2402 MHz 至 2480 MHz 之 2.4 GHz 工業、科學與醫療（ISM）免授權頻段，並將整體頻段劃分為 40 個間距為 2 MHz 的射頻通道（Channels）。在通道配置上，BLE 將通道分為 37 個資料通道（Data Channels）與 3 個廣播通道（Advertising Channels），其中資料通道主要用於裝置建立連線後的雙向資料傳輸，廣播通道則用於設備探索、廣播封包傳送與初始連線建立。

BLE 的三個廣播通道分別為 CH37、CH38 與 CH39，其對應頻率分別位於 2402 MHz、2426 MHz 與 2480 MHz，並未集中於相鄰頻段，而是分散配置於 2.4 GHz 頻段的不同位置，此配置可降低單一頻段受到干擾時對廣播程序造成的影響，提升裝置被掃描與發現的穩定性，如圖 2.1 所示。

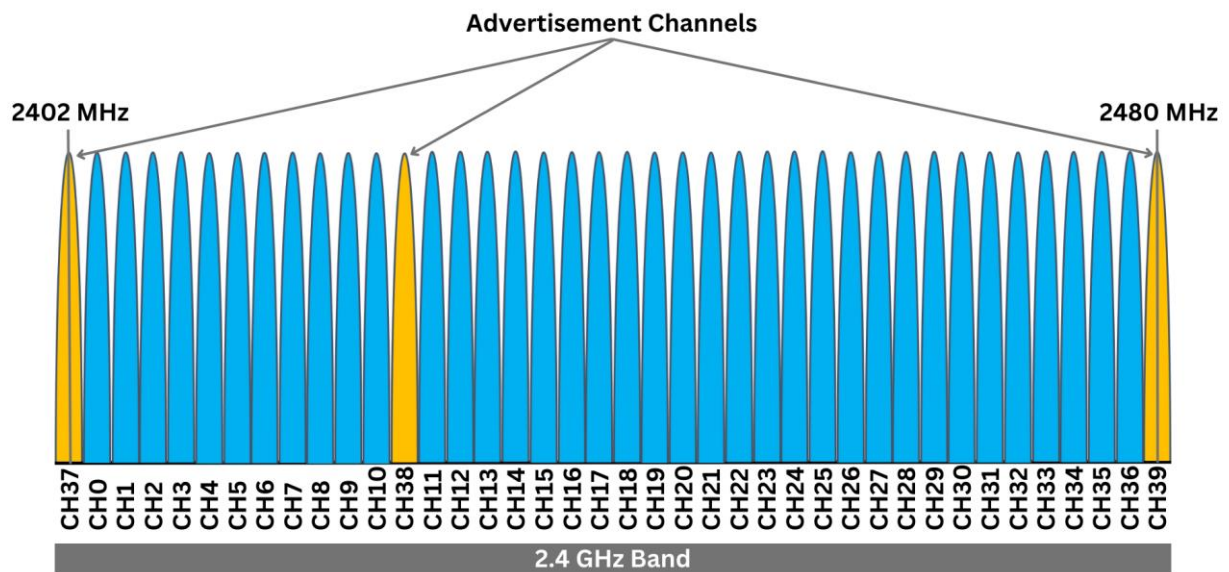


圖 2.1 BLE 廣播通道與資料通道分布[4]

BLE 的傳輸方式可概分為兩種，分別為廣播模式（Advertising mode）與連線模式（Connection-oriented mode）。在 BLE 的 Link Layer 中，裝置會依通訊流程切換於不同狀態，包括等待（Standby）、廣播（Advertising）、掃描（Scanning）、初始化（Initiating）及連線（Connected），透過這些狀態的轉換，BLE 裝置能先完成設備探索與連線建立，再依實際需求進行後續資料傳輸，藉此支援不同應用情境下的通訊需求，如圖 2.2 所示。

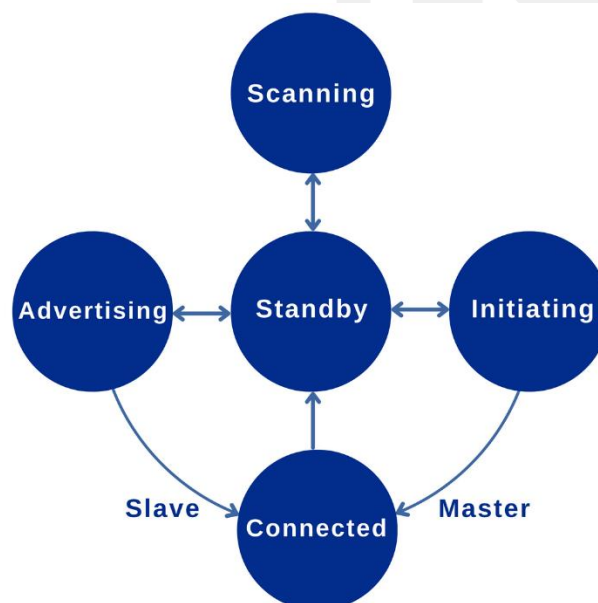


圖 2.2 BLE link layer 狀態轉換[4]

2.1.2 廣播模式 (Advertising Mode)

在廣播模式中，設備的運作皆以待機狀態 (Standby) 作為切換樞紐，當設備具有發送資訊的需求時，會由待機狀態切換至廣播狀態 (Advertising) 並擔任廣播端 (Advertiser)，此時廣播端會使用 CH37(2402 MHz)、CH38(2426 MHz) 與 CH39(2480 MHz) 三個廣播通道，並依序傳送廣播封包。相對地，若設備欲接收周圍裝置所發出的資訊，則會由待機狀態切換至掃描狀態 (Scanning) 並擔任掃描端 (Scanner)，掃描端會週期性地監聽上述三個廣播通道，以接收周圍裝置的廣播資料，進而完成裝置發現與資訊收集。

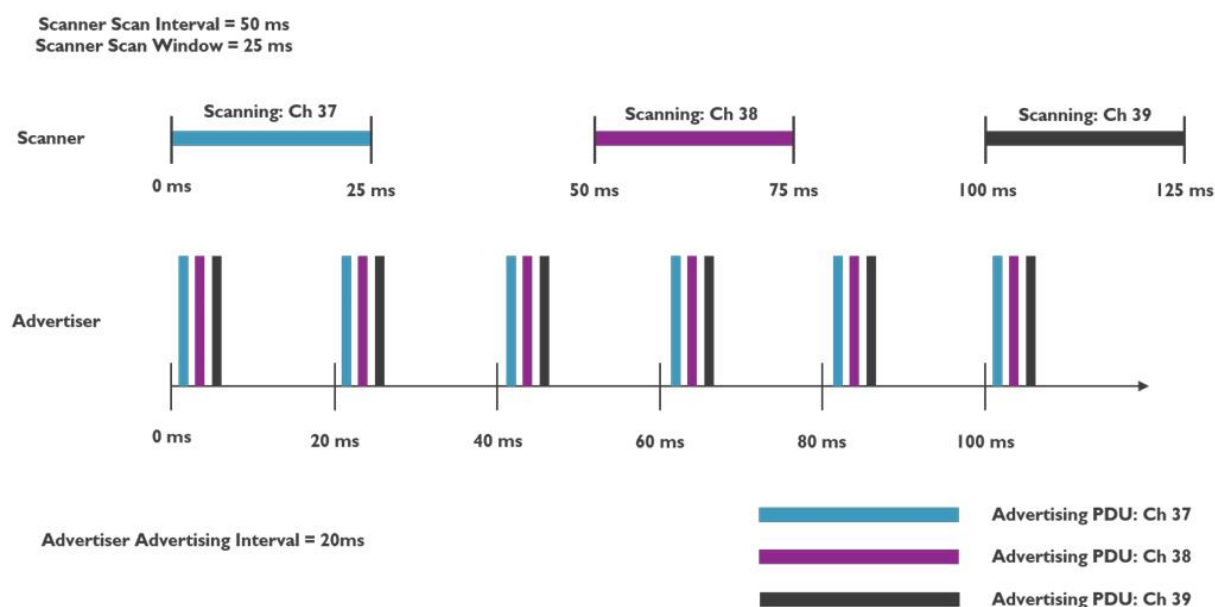


圖 2.3 BLE 廣播模式時序示意圖[5]

如圖 2.3 所示，廣播端依據設定之 Advertising Interval 週期性傳送封包，圖中廣播間隔為 20 ms，因此廣播端每隔 20 ms 便會依序於 CH37、CH38 與 CH39 上送出 Advertising PDU，掃描端則依據 Scan Interval 與 Scan Window 進行監聽，圖中 Scan Interval 為 50 ms，Scan Window 為 25 ms，表示掃描端每隔 50 ms 啟動一次掃描，並在單一通道上維持 25 ms 的監聽時間，掃描端會依序切換不同廣播通道進行監聽，因此廣播封包是否能被接收，取決於封包發送時間是否與掃描端的監聽視窗重疊，若兩者時間重疊，掃描端即可成功接收封包。

2.1.3 連線模式 (Connection-oriented Mode)

連線模式主要用於兩個 BLE 裝置完成連線建立後的資料交換，裝置角色可分為 Master (Central) 與 Slave (Peripheral)，其中 Master 負責連線時序的管理與同步，Slave 則依照 Master 所安排的時間點進行資料收發，該模式會使用其餘 37 個資料通道進行傳輸，透過自適應跳頻 (Adaptive Frequency Hopping, AFH) 在不同通道間切換，並根據通道品質避開干擾嚴重的頻道，選擇較穩定的通道進行傳輸，以降低通道干擾對通訊造成的影響。

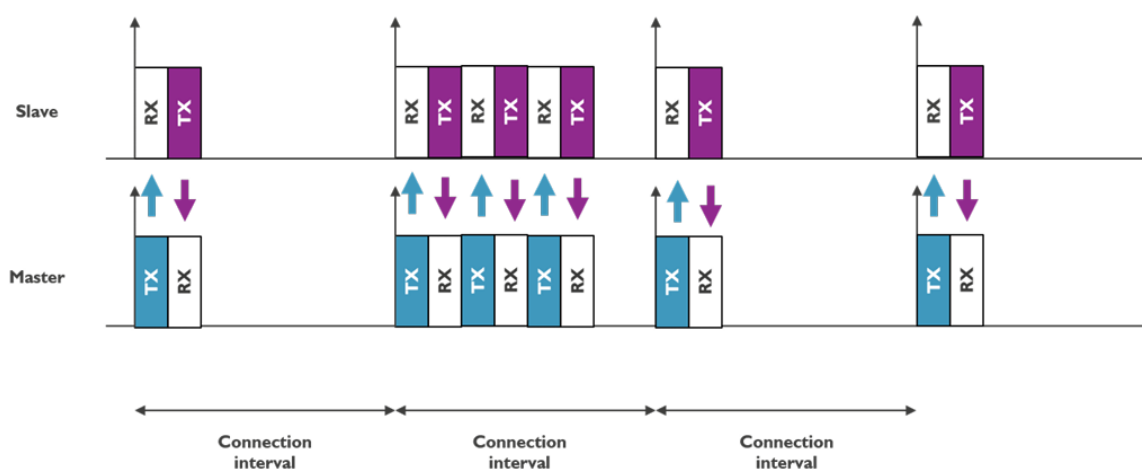


圖 2.4 連線模式資料傳輸時序示意圖[5]

如圖 2.4 所示，連線建立後，雙方會依據 Connection Interval 週期性進行資料交換，每一個 Connection Interval 內可包含一次或多次資料傳輸事件，由 Master 先傳送封包，Slave 接收後再回傳資料封包。圖中 TX 代表資料傳送，RX 代表資料接收，兩者在時間上呈現對應關係，也就是 Master 傳送時 Slave 進行接收，Slave 回傳時 Master 則切換為接收狀態，當該次連線事件結束後，雙方可進入低功耗狀態，並於下一個 Connection Interval 到來時再次喚醒進行傳輸。

2.1.4 BLE 排程機制

在 BLE 建立連線時，Advertiser 會週期性傳送廣播封包，而 Scanner 會在廣播通道上進行掃描。當 Scanner 接收到 Advertiser 的封包後，會在 T_{IFS} ($150 \mu s$) 後回

應 CONNECT_IND 封包，以完成連線建立。此時，原本負責掃描並發起連線的 Scanner 會成為連線中的 Master，而原本發送廣播封包的 Advertiser 則會成為 Slave。CONNECT_IND 中包含 WinOffset 與 WinSize 等參數，這些參數會影響第一個連線事件的起始時間，也就是 Anchor Point (AP)。

連線建立後，Master 與 Slave 會依照固定的 Connection Interval (CI) 週期性喚醒並交換資料。每一次資料交換的時間區段稱為 Connection Event (CE)，而 CE 的開始時間稱為 Anchor Point (AP)。因此，CI 會影響資料交換的頻率，而 AP 則決定 CE 在時間軸上的位置。

此外，第一個 AP 不會在 CONNECT_IND 結束後立即產生，而是會先經過固定的 transmitWindowDelay，其值固定為 1.25 ms，之後再根據 WinOffset 決定第一個 AP 的偏移位置，而 WinSize 則表示 AP 可能出現的時間範圍，若將 WinSize 設為 0，代表傳輸視窗的範圍被縮小為單一時間點，因此新的 Anchor Point (AP) 可以被設定在較明確且固定的位置，有助於 Master 精準安排後續的 connection event，如圖 2.5 所示。

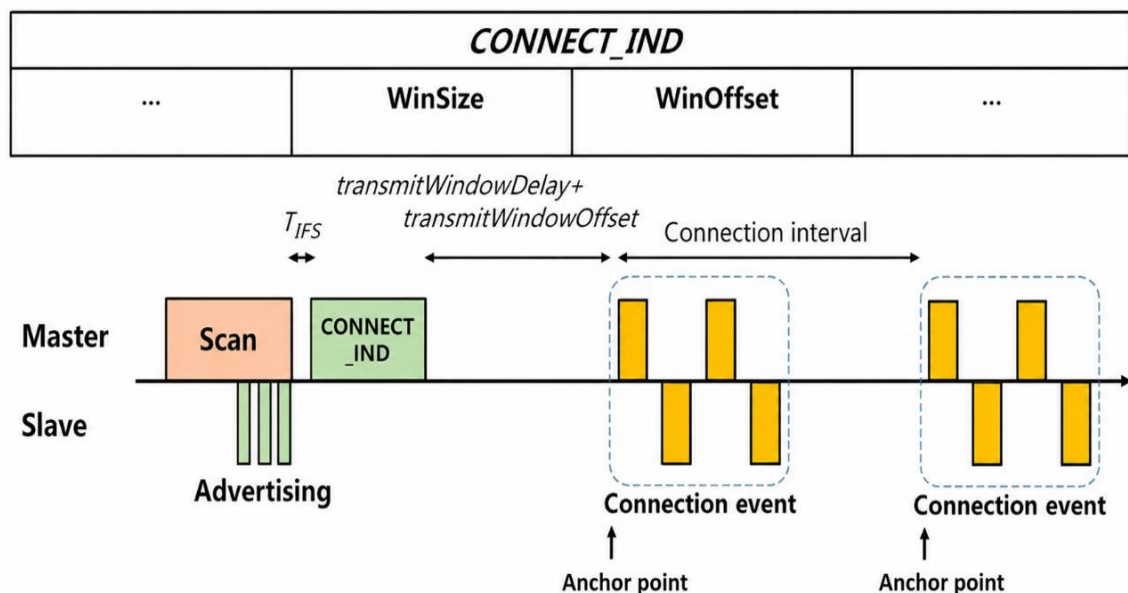


圖 2.5 BLE 連線建立與 Connection event 時間安排示意圖[6]

2.2 Time Division Multiple Access (TDMA)

分時多工（Time Division Multiple Access, TDMA）是一種讓多個使用者或裝置共用同一傳輸媒介的通訊技術，其主要概念是將時間切割成多個時槽（Time Slot），並將不同的時槽分配給不同的使用者，每個使用者只能在自己被分配到的時槽內進行資料傳輸，其餘時間則處於等待或非傳輸狀態，因此可以避免多個使用者在同一時間傳送資料而造成訊號重疊或封包碰撞。

在 TDMA 架構中，系統會依照排程安排各使用者的傳輸順序，使不同使用者能夠在不同時間點輪流使用通道，由於同一時間內只允許一個使用者進行傳輸，因此能降低通道競爭與干擾發生的機率，提升資料傳輸的穩定性。

2.3 Bluetooth Mesh 網狀網路

Bluetooth Mesh 是一種建立在 BLE 技術上的網狀網路架構，主要目的在於改善傳統 BLE 在傳輸距離與連線數量上的限制，透過 Mesh 架構，網路中的節點可以不只依靠直接連線進行通訊，也能透過其他節點協助轉發訊息，使資料能夠傳送到更遠的裝置，因此 Bluetooth Mesh 適合應用於智慧家庭、工業監控、感測網路與物聯網等需要大量節點通訊的場景。

在 Bluetooth Mesh 中，資料通常會透過多跳傳輸的方式進行傳遞，當來源節點無法直接將訊息送到目的節點時，封包可以經由鄰近節點逐步轉送，最後到達目標裝置，這種架構具有較高的網路彈性，因為資料不只依賴單一路徑傳輸，即使部分節點發生故障，訊息仍有機會透過其他路徑繼續傳遞，進而降低單一節點失效對整體網路造成的影響。

Bluetooth Mesh 的傳輸方式大致可分為 Flooding Mesh 與 Routing Mesh，Flooding Mesh 是以廣播方式傳送訊息，當節點收到封包後，會再將訊息轉發給附近的其他節點，使封包能夠快速擴散到整個網路，這種方式的優點是架構簡單、建立速度快，且不需要事先規劃完整的傳輸路徑，因此適合用於需要快速傳播訊息的環境，不過 Flooding Mesh 也容易產生重複傳輸，造成通道負載增加、封包碰撞率提高以及能源消耗上升，所以實

際系統通常會加入轉發次數限制、重複封包過濾或 Relay 節點控制等機制，以避免網路過度壅塞。

Routing Mesh 是一種以連線模式為基礎的網狀網路傳輸方式，節點在進行資料傳輸前，會先與鄰近節點建立連線，並依照網路拓樸或路由資訊，將資料逐步轉送至目的節點，與 Flooding Mesh 透過廣播方式擴散封包不同，Routing Mesh 的傳輸方向較明確，封包會沿著特定路徑進行傳遞，因此能減少不必要的重複廣播，降低通道負載、封包碰撞與能源消耗，使資料傳輸更穩定且有秩序，由於資料是透過已建立的連線進行交換，因此系統也較容易控制封包流向，並可依照節點狀態或傳輸需求調整路由，對於需要穩定傳輸、降低能耗或進行 QoS 控制的應用而言，Routing Mesh 具有較高的實用性，不過，此方式也需要維護節點之間的連線關係與路由資訊，當節點加入、離開或網路拓樸改變時，系統必須重新調整傳輸路徑，因此設計上較為複雜，此外，連線建立需要額外時間，若資料需要經過多個中繼節點才能抵達目的端，也可能增加整體傳輸延遲。

2.4 Nordic nRF52840-DK

本研究採用 Nordic Semiconductor 公司所開發的 nRF52840-DK 開發板作為 BLE 實驗設備，如圖 2.6 所示。nRF52840-DK 是以 nRF52840 SoC 為核心的開發平台，支援 Bluetooth Low Energy、Bluetooth Mesh、Thread、Zigbee、ANT、IEEE 802.15.4 以及 2.4 GHz 無線通訊等多種協定，因此適合用於低功耗無線通訊與物聯網相關應用之開發與測試。

nRF52840 SoC 採用 32 位元 ARM Cortex-M4 處理器，具備 64 MHz 運作頻率，並內建充足的 Flash 與 RAM 空間，可支援較複雜的嵌入式應用，且 nRF52840 也提供多種周邊介面，例如 USB、SPI、QSPI、I2C、I2S 與 PDM 等，使其能夠連接不同感測器或外部裝置，增加實驗與系統開發的彈性。

由於 nRF52840-DK 同時具備低功耗、多協定支援以及良好的開發擴充性，因此適合作為 BLE Mesh 網路實驗中的節點設備，本研究透過多個 nRF52840-DK 開發板建

立實驗環境，並用於後續節點連線、封包傳輸與網路效能測試。

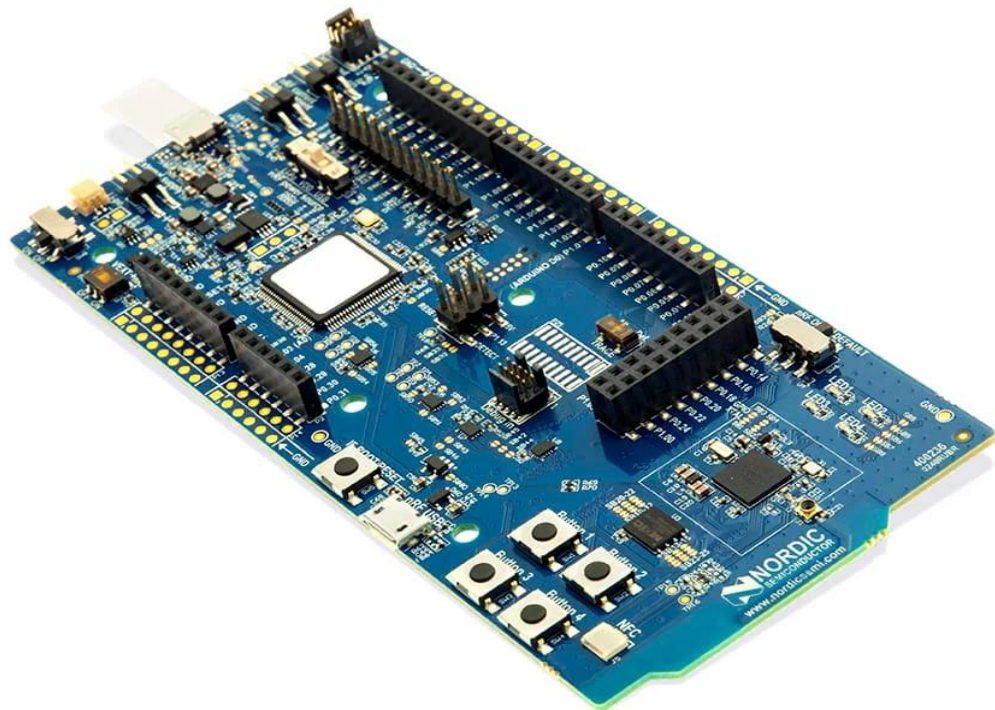


圖 2.6 Nordic nRF52840-DK 開發板[8]

2.5 FruityMesh 介紹

FruityMesh 是一套以 BLE 連線模式為基礎的 Mesh 網路框架，主要透過節點之間建立 BLE 連線來形成可進行多跳傳輸的網路，與單純以廣播方式擴散封包的 Flooding Mesh 不同，FruityMesh 會讓節點彼此維持連線關係，並透過中繼節點協助轉送資料，使封包能夠由來源節點逐步傳遞至目的節點，藉此延伸 BLE 裝置原本有限的通訊範圍。

在 FruityMesh 中，節點除了傳送自身資料外，也可能作為中繼節點協助其他節點轉送封包，因此適合用於多節點、低功耗與長時間運作的物聯網環境，其主要運作機制包含 Clustering 與 Self-Healing，前者負責網路拓樸的建立，後者則用於節點斷線後的拓樸恢復。

2.5.1 Clustering 網路形成機制

Clustering 是 FruityMesh 建立網路拓樸的主要機制，當節點啟動後，會開始搜尋周圍可連線的節點，並嘗試與鄰近節點建立 BLE 連線，隨著節點之間逐漸完成連線，網路中會先形成數個不同的 cluster，之後再透過 cluster 之間的合併，逐步形成一個包含所有節點的完整網路。

在 FruityMesh 的原始設計中，節點選擇連線對象時會將 cluster size 作為重要依據，較大的 cluster 具有較高的連線決定權，因此會傾向選擇並合併較小的 cluster，使分散的節點逐漸整合至同一個網路中。最終，FruityMesh 所形成的拓樸通常會呈現類似 Spanning Tree 的樹狀架構，節點之間以父子關係連接，並透過既有連線進行多跳封包轉送。

為了避免網路中產生迴圈，FruityMesh 不會讓節點與相同 clusterId 的節點建立連線，因此可降低重複路徑與 loop 發生的可能性，透過此種樹狀拓樸，封包傳輸方向較為明確，但拓樸形成結果也會影響後續封包傳輸路徑與中繼節點負載。

2.5.2 Self-Healing 拓樸恢復機制

Self-Healing 是 FruityMesh 用於維持網路連通性的自我修復機制，當節點因斷線、重啟或訊號不穩而離開原本網路時，系統會使該節點重新進入掃描與連線流程，並嘗試與通訊範圍內的鄰近節點重新建立連線。

透過 Self-Healing 能在部分節點失效後自動修復樹狀拓樸，使孤立節點重新加入網路，並恢復資料傳輸路徑，此機制可提升網路的容錯能力，但其恢復時間會受到節點數量、掃描時間、鄰近節點分布與連線選擇流程等因素影響。

2.5.3 Nordic SoftDevice

Nordic SoftDevice 是 Nordic Semiconductor 提供的無線通訊協定堆疊，主要負責處理 nRF 系列晶片中的 BLE 通訊功能。SoftDevice 將 BLE 協定中較底層且即時性要

求較高的部分進行封裝，例如廣播、掃描、連線建立、連線維持與封包收發，使開發者能透過 API 進行應用程式開發，而不需要自行實作完整的 BLE 協定流程。

在系統運作上，SoftDevice 位於硬體與應用程式之間，負責管理無線通訊相關事件，而應用程式則可在其上層實作資料處理與節點控制邏輯。由於 BLE 對封包交換時間具有較高要求，SoftDevice 能協助系統穩定處理連線與資料傳輸，並作為後續 FruityMesh 應用執行時的重要基礎。

2.6 FruityMesh 傳輸效能改善方法

為改善 FruityMesh 在多跳傳輸過程中可能產生的封包重複傳送、角色衝突、資料匯流壅塞與傳輸延遲等問題，相關研究已針對傳輸方向、連線啟用方式、拓樸建立流程與連線參數配置進行改善。本節整理與本研究相關之 FruityMesh 傳輸效能改善方法，包含 Destination-Oriented Transmission (DOT)、Destination-Oriented Switchable Transmission (DOST)、Sink-oriented 拓樸建立機制，以及階層式 Connection Interval 調整機制，作為後續研究問題定位之基礎。

2.6.1 Destination-Oriented Transmission (DOT)

Destination-Oriented Transmission (DOT) 主要用於改善 FruityMesh 原生傳輸中封包轉送方向不明確的問題。在原始傳輸方式中，節點接收到封包後可能將封包傳送至多個鄰近節點，雖然可增加封包擴散機會，但也容易造成重複傳輸、頻寬浪費與封包碰撞。

DOT 透過樹狀拓樸中的 Level 層級資訊，使封包依照目的端方向進行轉送。當封包需要傳送至 Sink 或 Root 節點時，節點會優先將封包往 Level 較低、較接近目的端的方向傳遞，避免封包被轉送至與目的端無關的路徑。透過此方式，DOT 可減少不必要的封包傳輸，降低網路負載，並提升多跳傳輸效率 [3]。

2.6.2 Destination-Oriented Switchable Transmission (DOST)

DOT 透過樹狀拓樸中的 Level 層級資訊，使封包依照目的端方向進行轉送。當封包需要傳送至 Sink 或 Root 節點時，節點會優先將封包往 Level 較低、較接近目的端的方向傳遞，避免封包被轉送至與目的端無關的路徑。透過此方式，DOT 可減少不必要的封包傳輸，降低網路負載，並提升多跳傳輸效率 [3]。

Destination-Oriented Switchable Transmission (DOST) 針對此問題，透過連線啟用與禁用控制，使節點在特定時間內只處理部分連線的傳輸。當某一方向的連線被啟用時，其他可能造成衝突的連線會暫時停用，待下一個週期再進行切換。此方式可錯開不同連線的傳輸時間，降低角色衝突與封包重傳，並提升 FruityMesh 在多跳環境中的傳輸穩定性 [3]。

2.6.3 Sink-oriented 拓樸建立機制

FruityMesh 原生 Clustering 機制雖能自動形成樹狀拓樸，但無法保證資料匯集端 Sink 一定位於整體拓樸的 Root 位置。若 Sink 位於非根節點，封包在傳送至 Sink 的過程中可能需要經過較長路徑，甚至造成資料匯流方向不一致，進而影響整體傳輸效率。

為改善此問題，Sink-oriented 拓樸建立機制將 Sink 節點固定於樹狀拓樸的 Root 角色，使其他節點能依照父子關係逐步將封包轉送至 Sink。此方法可使資料傳輸方向更加明確，並減少因 Sink 位置不固定而產生的額外轉送與重傳問題。此外，當 Sink 節點斷線後重新加入網路時，該機制也能協助其恢復 Root 角色，維持整體拓樸的資料匯流一致性 [2]。

2.6.4 階層式 Connection Interval 調整機制

在樹狀拓樸中，越靠近 Sink 的節點通常需要負責轉送更多來自下層節點的封包，因此上層節點容易形成資料匯流瓶頸。若所有節點皆使用相同的 Connection Interval (CI)，靠近 Sink 的節點可能因傳輸機會不足而產生封包堆積，導致延遲增加、封包抵

達率下降或重傳率提高。

階層式 Connection Interval 調整機制依據節點距離 Sink 的層級配置不同 CI。越接近 Sink 的節點會使用較短 CI，使其 Connection Event 發生頻率較高，進而取得更多傳輸機會；距離 Sink 較遠的節點則可使用較長 CI，以降低不必要的傳輸頻率與能耗。透過此方式，可緩解上層中繼節點的傳輸負載，並改善封包抵達率、延遲與重傳率 [2]。

2.6.5 傳輸效能改善方法之限制與研究切入點

綜合上述，DOT 主要改善封包轉送方向，DOST 透過連線啟用與禁用降低角色衝突，Sink-oriented 拓樸建立機制使資料匯集方向更明確，而階層式 CI 調整則針對靠近 Sink 的節點提供更多傳輸機會。上述方法分別從封包路徑、連線控制、拓樸結構與連線參數等角度改善 FruityMesh 傳輸效能。

然而，當節點發生重啟、斷線或 Sink 重新加入網路時，系統仍需重新掃描鄰近節點、收集連線資訊並完成連線決策，因此拓樸恢復時間仍可能受到影響。此外，在高流量情境下，即使透過 CI 調整增加部分節點的傳輸機會，重要封包仍可能與一般封包競爭相同的傳輸資源，導致關鍵資料的延遲與抵達率受到影響。因此，後續章節將分別針對拓樸恢復效率與高流量下的服務品質進行探討。

第三章 BLE Mesh 網路韌性與服務品質強化方法

3.1 研究問題與設計目標

FruityMesh 透過 Clustering 與 Self-Healing 機制建立並維持樹狀拓樸，使 BLE 節點能在多跳環境下進行資料傳輸。前人研究已針對 FruityMesh 的傳輸效率與拓樸結構進行改善，例如透過 DOT 與 DOST 降低重複傳輸與角色衝突問題，並透過 Sink-oriented 拓樸建立與階層式 Connection Interval (CI) 調整，使資料能更有效地朝向 Sink 節點匯流。然而，在節點重啟、Sink 異常斷線或高流量傳輸情境下，FruityMesh 仍存在拓樸恢復時間過長與關鍵封包傳輸品質不穩定等問題，因此仍有進一步改善空間。

3.1.1 拓樸恢復延遲問題

在 FruityMesh 網路中，當節點發生 reset、斷線或重新加入網路時，原有的暫存連線資訊會消失，系統必須重新進行掃描、收集鄰近節點資訊、選擇連線對象並完成連線建立流程。雖然 Self-Healing 機制能使節點重新加入網路，但其流程主要依賴即時掃描與重新決策，因此在節點數量較多或可連線候選節點較複雜時，拓樸恢復時間可能增加。

對於 Sink 節點而言，拓樸恢復時間過長會造成更明顯的影響，由於 Sink 通常負責資料匯集，若 Sink 在斷線後無法快速恢復其網路位置，可能導致下層節點的資料傳輸暫時中斷，進而影響整體網路的服務穩定性。

3.1.2 高流量下封包傳輸品質問題

除了拓樸恢復問題外，FruityMesh 在高流量情境下也可能面臨傳輸資源競爭，當多個節點同時產生大量封包時，封包會競爭有限的 BLE 傳輸頻寬與連線事件資源。即使透過階層式 CI 調整增加部分節點的傳輸機會，當總流量超出可用頻寬時，重要封包仍可能受到一般封包影響，導致傳輸延遲增加或封包抵達率下降。

在實際應用中，並非所有資料都具有相同重要性。例如控制訊息、狀態回報或關鍵

感測資料，通常比一般週期性資料具有更高的即時性與可靠性需求，在高流量環境下，若系統無法區分封包重要性，可能使關鍵資料與一般資料使用相同傳輸機會，進而降低服務品質。

3.2 BLE Mesh 快速拓樸重建機制設計

為改善節點於 reset 後之拓樸重建延遲問題，本研究提出一種非揮發性歷史鄰居快速式快速拓樸重建機制（Non-Volatile Historical Neighbor Cache-based Fast Topology Recovery Mechanism）。本機制之核心設計理念為，在不修改 FruityMesh 原有拓樸建立與自我修復機制之情況下，引入歷史資訊輔助決策，使節點於重啟後能快速重新建立與既有網路之連線關係，進而縮短拓樸重建所需時間。

本研究所提出之快速拓樸重建機制，其整體流程如圖 3-1 所示，主要可分為 reset、NVM Load & Validation、Fast Decision Path、FruityMesh Flow 以及 Cache Update & Persist 五個階段。



圖 3-1 快速拓樸重建機制之系統總體流程圖

當節點發生 reset 並重新啟動後，系統首先進入初始化階段，隨即從非揮發性記憶體（NVM）中載入先前儲存之歷史鄰居紀錄表，並進行資料有效性驗證（NVM Load & Validation），以確保載入之資訊未受損且仍具參考價值。

完成驗證後，系統進入快速決策階段（Fast Decision Path）。在接收周圍節點所廣播之 JOIN_ME 封包時，會即時比對其來源是否存在於歷史紀錄表中；若命中，則視為具備歷史連線成功經驗之候選節點，系統將優先對其進行連線決策與嘗試，以縮短傳統 decision window 所需之等待時間。反之，若未命中歷史紀錄，系統則不啟用快速決策機制，而是依照 FruityMesh 原有機制進行標準 decision window 等待，並於等待結束後繼續執行既有之 JOIN_ME 收集、節點評分（scoring）、連線選擇、handshake 與後續 Self-Healing 等程序，以完成拓樸建立。

無論是否命中歷史紀錄，節點最終皆係沿用 FruityMesh 原生拓樸建立流程，兩者差異僅在於，當命中歷史紀錄時，系統可根據既有歷史資訊提前做出連線決策，因而無須完整等待原始 decision window；若未命中，則維持原先 FruityMesh 所定義之等待時間與決策方式，因此整體協定流程並未改變。

當連線結果確立後，系統皆會進入快取更新階段（Cache Update & Persist），並根據此次連線經驗更新歷史鄰居紀錄表，再將其寫回非揮發性記憶體，以供後續節點重啟時使用。本研究採用固定大小之快取設計，每個節點僅維護有限數量之鄰居紀錄（例如三筆），並透過動態替換機制持續保留較具參考價值之歷史資訊。

透過上述設計，本機制在不改變 FruityMesh 原有協定架構之前提下，僅於拓樸重建初期針對 decision window 之等待行為進行優化，使節點在命中歷史紀錄時得以更快進入既有 FruityMesh 流程，而未命中時則完全維持原始運作方式，因此在最壞情況下之重建時間不會劣於既有機制，兼顧效能提升與系統相容性。

3.2.1 歷史鄰居快取資料結構與載入驗證機制

為使節點具備跨重啟之歷史記憶能力，本研究於節點端設計一套歷史鄰居快取機制（Historical Neighbor Cache），用以保存過去曾成功參與拓樸建立之高價值鄰居資訊。與直接儲存完整拓樸狀態之方式不同，本機制僅保留與快速重建決策高度相關之關鍵摘要資料，使系統在節點 reset 後能以較低的儲存與計算成本，快速建立具參考價值之候選節點集合，同時降低過期或低品質資訊對後續決策造成誤導之風險。

在資料結構設計上，本研究定義每一筆 HistoricalNeighborEntry 包含多項關鍵欄位：節點識別碼(nodeId)、歷史角色資訊(lastRole)、最後一次量測之訊號強度(lastRSSI)、歷史平均訊號強度 (avgRSSI)、對應之 hopsToSink、成功次數 (successCount)、失敗次數 (failureCount)、最近有效時間 (lastValidTimeDs)、有效旗標 (validFlag)，以及用於資料完整性驗證之 checksum。其中，nodeId 用於唯一識別鄰居節點；successCount 與 failureCount 反映歷史連線成功率與穩定性；avgRSSI 與 hopsToSink 分別描述鏈路品

質與拓樸方向性；lastValidTimeDs 可用於評估資料新鮮度；而 validFlag 與 checksum 則確保從非揮發性記憶體讀回之資料具備基本可信度。透過上述欄位設計，使歷史快取同時兼顧連線可靠度、鏈路品質與拓樸位置資訊，作為後續快速決策機制的重要依據。

由於節點於 reset 後無法依賴 RAM 中的暫存狀態，系統啟動時須先自非揮發性記憶體（Non-Volatile Memory, NVM）載入歷史鄰居快取資料，並執行完整之資料一致性與有效性驗證流程。其整體流程如圖 3-2 所示。

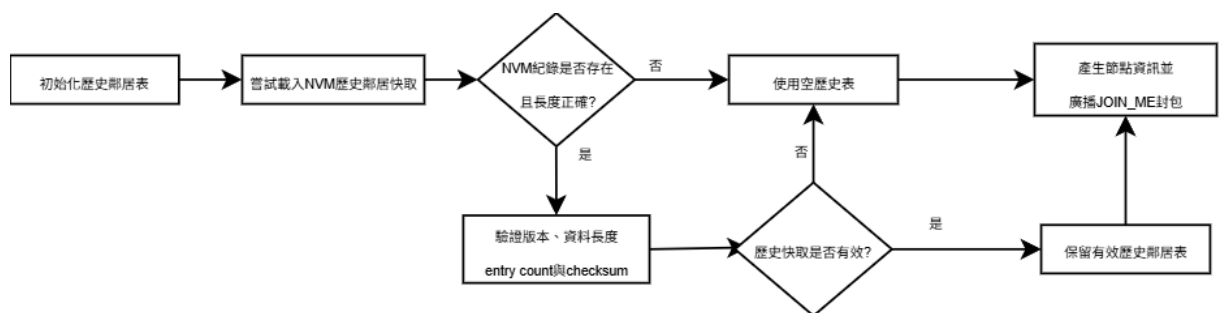


圖 3-2 歷史鄰居快取載入與驗證流程圖

首先，系統初始化歷史鄰居表結構，並檢查紀錄儲存功能是否啟用；若未啟用或 NVM 中不存在對應資料，則直接回傳空表。接著，系統讀取指定之歷史紀錄區塊，並檢查其資料長度是否符合預期之結構大小，以避免讀取不完整或異常資料。隨後，系統驗證資料版本（version）是否與目前系統定義一致，以確保資料格式相容性，同時檢查 entryCount 是否超出系統所設定之最大容量限制（MAX_HISTORY_ENTRIES），以避免不合理資料導致系統錯誤。

在完成整體結構層級之檢查後，系統進一步針對每一筆 entry 進行逐項驗證，包括 nodeId 合法性檢查與 checksum 正確性驗證；若任一筆資料驗證失敗，則視為整體資料不可信，系統將放棄該次載入結果並回傳空表，以確保後續決策不受錯誤歷史資訊影響。當所有檢查皆通過時，系統方將載入結果正式指定為有效之 historicalNeighborTable，並重新計算實際有效 entry 數量，作為後續快速決策機制之輸入，其具體實作流程如 Algorithm 1 所示。

3.2.2 快速決策機制 (Fast Decision Path)

在既有 FruityMesh 拓樸建立流程中，節點接收到 JOIN_ME 封包後，會依照目前網路狀態計算原始候選分數，用以決定後續應優先處理之連線對象。然而，該分數僅反映當前瞬時資訊，無法利用節點在 reset 前已累積之鄰居經驗。為此，本研究提出一套啟發式評分模型 (Heuristic Scoring Model)，將歷史鄰居資訊轉換為額外的 ranking bonus，並疊加於原有 FruityMesh 分數之上，使較有可能成功重建的鄰居獲得較高優先權。

本研究之紅利分數設計遵循三項原則。第一，**穩定性優先**：若某節點過去曾多次成功參與重建，或曾作為穩定父節點，則應給予較高加權；若連線失敗次數過高，則需明顯扣分。第二，**訊號一致性校驗**：若目前收到的 RSSI 與歷史觀測值相近，代表該鄰居之無線條件未發生明顯惡化，因此可再給予額外獎勵。第三，**拓樸方向引導**：對於 hopsToSink 較小之節點，應提高其排名，以促使 reset 後之快速重建仍朝向較接近 Sink 之方向收斂。

Input: 當前收到之 JOIN_ME 封包 packet

Output: 歷史加權紅利 bonus

1: entry $\leftarrow$ FindHistoricalNeighborEntry(packet.payload.sender)

2: if entry 不存在 then

3: return 0

4: end if

5: if IsHistoricalNeighborEntryValidForRanking(entry) = false then

6: return 0

7: end if

8: bonus $\leftarrow$ 0

```

9: bonus  $\leftarrow$  bonus + 200  $\times$  entry.successCount
10: bonus  $\leftarrow$  bonus - 300  $\times$  entry.failureCount
11: bonus  $\leftarrow$  bonus + 8  $\times$  (entry.avgRSSI + 100)

12: if entry.hopsToSink  $\geq$  0 then
13:     bonus  $\leftarrow$  bonus - 120  $\times$  entry.hopsToSink
14: end if

15: if entry.lastRole = PARENT then
16:     bonus  $\leftarrow$  bonus + 250
17: end if

18: if packet.rssi  $\geq$  entry.lastRSSI - 10 then
19:     bonus  $\leftarrow$  bonus + 80
20: end if

21: return bonus

```

若將演算法 3-2 抽象化表示，則可寫為：

$$\begin{aligned}
 \text{Bonus}_i &= W_s \cdot N_{\text{succ}} - W_f \cdot N_{\text{fail}} + W_r \cdot \text{RSSI}_{\text{avg}} - W_h \cdot \text{HopsToSink} + W_p \cdot \text{ParentBonus} + W_c \cdot \\
 &\text{ConsistencyBonus} \\
 \text{Bonus}_i &= W_s \cdot N_{\text{succ}} - W_f \cdot N_{\text{fail}} + W_r \cdot \text{RSSI}_{\text{avg}} - W_h \cdot \text{HopsToSink} + W_p \cdot \text{ParentBonus} + W_c \cdot \text{ConsistencyBonus}
 \end{aligned}$$

其中， N_{succ} 表示歷史成功次數， N_{fail} 表示歷史失敗次數， RSSI_{avg} 為平均訊號強度， HopsToSink 為拓樸跳數， ParentBonus 表示該鄰居是否曾作為穩定父節點，而 ConsistencyBonus 則用於檢查目前 RSSI 是否與歷史觀測相近。最終，系統將此一紅利值與 FruityMesh 原始分數整合為：

$AdjustedScore = OriginalScore + Bonus$
 $AdjustedScore = OriginalScore + Bonus$

如此一來，節點在 reset 後不僅考慮當前封包之瞬時資訊，也能利用過去已證實有效之鄰居經驗，提高快速重建之成功率

3.2.3 非同步決策攔截與計時器優化

在原始 FruityMesh 流程中，節點通常需經歷一段 discovery 與 decision window 後，才會正式進入 Master/Slave 決策程序。此設計有助於節點蒐集足夠之候選資訊，但在 reset 情境下，也會導致節點即使已遇到高可信鄰居，仍須等待完整的掃描期結束後才能採取行動。為縮短此一等待成本，本研究提出非同步決策攔截機制（Asynchronous Decision Interception），使系統在偵測到高可信候選時，得以提早進入既有決策流程。

其核心想法並非繞過 FruityMesh 原始決策與握手機制，而是於 OnJoinMeReceived() 收到封包並完成 AdjustedScore 計算後，若該候選分數高於預設門檻，則透過調整決策時戳（如 lastDecisionTimeDs）方式，人為製造「決策視窗已屆期」的條件，使系統可立即執行既有之 Master/Slave 選擇程序。換言之，本研究優化的是「何時啟動決策」，而不是「直接跳過決策」。

此機制可表示為演算法 3-3。

Algorithm 3-3 FastDecisionInterceptionOnJoinMe()

Input: JOIN_ME 封包 packet, OriginalScore, HistoricalBonus

Output: 是否提早觸發既有決策程序

- 1: AdjustedScore $\leftarrow$ OriginalScore + HistoricalBonus
- 2: if AdjustedScore < FAST_TRIGGER_THRESHOLD then
- 3: 保持原始 decision window

4: return NORMAL_PATH

5: end if

6: if 歷史 entry 不存在 或 未通過有效性驗證 then

7: 保持原始 decision window

8: return NORMAL_PATH

9: end if

10: 將 lastDecisionTimeDs 調整為已滿足決策條件之時間

11: return FAST_TRIGGERED

透過此作法，系統可在高可信候選者出現時縮短被動等待時間，提早進入 FruityMesh 既有之 DetermineBestCluster... 決策流程。需要強調的是，本研究並未破壞後續的 Master/Slave 選擇、握手與 Self-Healing 程序，而僅是讓該程序更早被觸發。因此，此機制與原始 FruityMesh 邏輯是相容的，可視為一種 reset 情境下的時間優化技術。

3.2.4 歷史鄰居表更新與持久化策略

歷史鄰居快取若僅在載入後使用，而不根據連線結果進行更新，則無法反映後續環境變動。因此，本研究在每次快速重建成功或失敗後，皆透過 UpsertHistoricalNeighborEntry() 更新該候選之歷史紀錄，並於必要時透過 PersistHistoricalNeighborTableIfNeeded() 將更新結果寫回 NVM。

成功重建時，系統提高 successCount、清除 failureCount、更新 lastValidTimeDs，並維持 validFlag 為有效狀態；失敗時則增加 failureCount，若該值累積至門檻，則將其標記為暫時不可用，以避免後續持續重試同一個不穩定鄰居。此一機制可視為一種帶有退避（backoff）特性的信用評等更新策略。

為降低 flash 擦寫頻率，本研究不採用每次更新即立即寫回之方式，而是採用 dirty flag 與最小寫入時間間隔控制持久化時機。當歷史表被標記為 dirty，且目前無寫入作業進行中，並且距離上次寫入嘗試已超過最小間隔時，系統才會真正觸發 NVM 寫入。此種設計可在保存 reset 後記憶能力的同時，兼顧嵌入式平台對非揮發性儲存體壽命之要求。

Algorithm 3-4 UpdateAndPersistHistoricalNeighborTable()

Input: candidate, reconstruction result

Output: 更新後之歷史鄰居表與持久化狀態

```
1: if reconstruction result = SUCCESS then
2:   candidate.successCount  $\leftarrow$  candidate.successCount + 1
3:   candidate.failureCount  $\leftarrow$  0
4:   candidate.lastValidTimeDs  $\leftarrow$  CurrentTime
5:   candidate.validFlag  $\leftarrow$  true
6: else
7:   candidate.failureCount  $\leftarrow$  candidate.failureCount + 1
8:   if candidate.failureCount  $\geq$  FAIL_THRESHOLD then
9:     candidate.validFlag  $\leftarrow$  false
10:  end if
11: end if

12: 重新計算 checksum
13: historicalNeighborDirty  $\leftarrow$  true

14: if 達到持久化條件 then
```

15: 限流後寫回 NVM

16: end if

17: return

由此可知，本研究之歷史鄰居快取並非靜態資料表，而是一個隨節點重建結果動態調整之信用記錄系統。其功能不僅在於 reset 後提供快速重建依據，也在於透過成功與失敗經驗的持續回饋，逐步保留較高品質之歷史鄰居集合。

3.3

3.3.1



第四章 實驗結果與分析

4.1 實驗環境

4.2

4.2.1

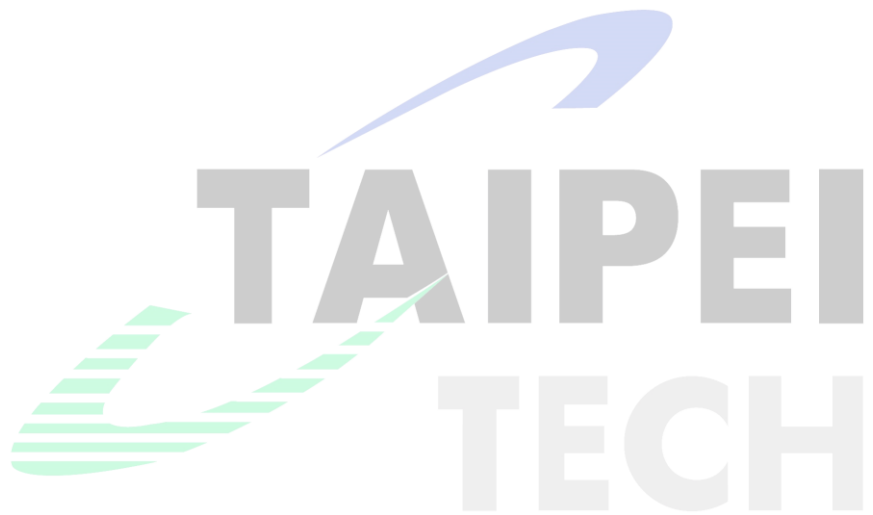
4.2.2

4.3

4.3.1

4.3.2

4.3.2.1



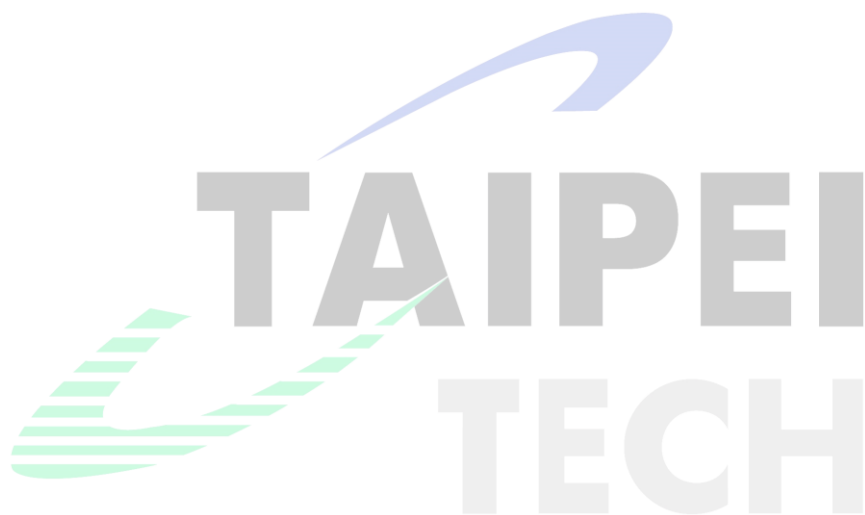
4.3.3

4.3.4

\

4.3.5

4.3.6



第五章 結論與未來展望

5.1 結論

5.2 展望



參考文獻

- [1] Mordor Intelligence, "Internet of Things (IoT) Market Size & Share Analysis - Growth Trends and Forecast (2025 - 2030)," Mordor Intelligence, 2025. [Online]. Available: <https://www.mordorintelligence.com/industry-reports/internet-of-things-iot-market>
- [2] 陳柏勳, "改善 FruityMesh 藍牙低功耗網狀網路傳輸效能的設計與實作," 碩士論文, 國立臺北科技大學資訊工程系, 2025.
- [3] 溫淇淼, "基於 fruitymesh 之藍牙低功耗網狀網路傳輸機制的設計以改善 qos," 碩士論文, 國立臺北科技大學資訊工程系, 2024.
- [4] RAKwireless Technology Limited, "BLE Introduction," RAKwireless Documentation Center. [Online]. Available: <https://docs.rakwireless.com/product-categories/software-apis-and-libraries/rui3/rui-ble-examples/>
- [5] M. Baert, J. Rossey, A. Shahid, E. De Poorter, and J. Hoebeke, "The Bluetooth mesh standard: An overview and experimental evaluation," *Sensors*, vol. 18, no. 8, p. 2409, 2018.
- [6] E. Park, H.-S. Kim, and S. Bahk, "BLEX: Flexible Multi-Connection Scheduling for Bluetooth Low Energy, " in *Proc. ACM/IEEE Int. Conf. Information Processing in Sensor Networks (IPSN)*, Nashville, TN, USA, May 18–21, 2021, pp. 1–15.
- [7] Bluetooth Special Interest Group, *Bluetooth Core Specification Version 5.0*, Dec. 2016. [Online]. Available: <https://www.bluetooth.com/specifications/specs/core-specification-5-0/>.
- [8] Nordic Semiconductor, "nRF52840 DK Documentation," Nordic Semiconductor Documentation. [Online]. Available: <https://docs.nordicsemi.com/>.
- [9] FruityMesh, "BlueRange Mesh Community Documentation," [Online]. Available: <https://bluerange.io/docs/bluerange-mesh-community/index.html>.